

GNU Make

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Writing makefiles	2
2	Variables	10
3	Patterns and substitutions	13
4	Make functions	16
5	Conditionals	19
6	User-defined functions	20
7	More on make	21
8	Header dependencies	22
9	Handling dependencies in a build	24

Writing makefiles

- What a rule looks like:

```
1 target ... : prerequisites ...
2     recipe
3     ...
4     ...
```

A target is usually the name of a file that is generated by a program; examples of targets are executable or object files. A target can also be the name of an action to carry out, such as ‘clean’

A prerequisite is a file that is used as input to create the target. A target often depends on several files.

A recipe is an action that make carries out. A recipe may have more than one command, either on the same line or each on its own line. Please note: you need to put a tab character at the beginning of every recipe line! This is an obscurity that catches the unwary. If you prefer to prefix your recipes with a character other than tab, you can set the **.RECIPEPREFIX** variable to an alternate character

Usually a recipe is in a rule with prerequisites and serves to create a target file if any of the prerequisites change. However, the rule that specifies a recipe for the target need not have prerequisites. For example, the rule containing the delete command associated with the target ‘clean’ does not have prerequisites.

rule, then, explains how and when to remake certain files which are the targets of the particular rule. make carries out the recipe on the prerequisites to create or update the target. A rule can also explain how and when to carry out an action.

- **The prereqs:** In a normal Make rule, prerequisites are used by make for timestamp-based rebuild decisions and build ordering.

```
1 main.o : main.c defs.h
2     cc -c main.c
```

Make checks

- Does main.o exist?
- Is main.c newer than main.o?
- Is defs.h newer than main.o?

If main.o does not exist, or if any prerequisite is newer than main.o, then make runs the recipe.

- **There are also order-only prerequisites::**

```

1  main.o : main.c | build
2      cc -c main.c -o build/main.o

```

The build prerequisite after | means: Make sure build exists before building main.o, but do not rebuild main.o just because build is newer.

Before building main.o, make sure there is a target or file-system entry named build.

Make treats targets as pathnames. A directory can be a target because it is also a file-system entry.

you would normally add a rule telling make how to create it.

```

1  build:
2      mkdir -p build

```

- **Target with no prereqs:** In make, whether a target runs depends mainly on whether the target file exists and whether it is out of date.

```

1  clean:
2      rm -rf *.o editor

```

Make asks: "Does a file named clean exist?". If no file named clean exists, the recipe runs. So,

```

1  make clean

```

Usually runs every time. But if a file named clean exists, then make may say

```

1  make: 'clean' is up to date

```

and not run the recipe. This is why command style targets are usually marked .PHONY

```

1  .PHONY: clean
2
3  clean:
4      rm -r *.o editor

```

.PHONY tells make: "clean is not a real file target. Always run its recipe when requested."

- **Goals:** Make does not run every rule in the makefile. When you run

```
1 make
```

make builds the **default goal**, which is normally the **first target** in the Makefile.

```
1 editor: main.o kbd.o command.o display.o insert.o search.o
   ↪ files.o utils.o
2 cc -o editor main.o kbd.o command.o display.o insert.o
   ↪ search.o files.o utils.o
3
4 main.o: main.c defs.h
5 cc -c main.c
6
7 clean:
8 rm -f *.o editor
```

Here, the first target is **editor**. So, running **make** means

```
1 make editor
```

make then builds editor and anything editor depends on.

- **all**: all is just a conventional target name used to mean: Build the main thing, or build everything important.

```
1 all: editor
```

means to build all, first build editor.

```
1 .PHONY: all clean
2
3 all: editor
4
5 editor: main.o kbd.o command.o
6 cc -o editor main.o kbd.o command.o
7
8 main.o: main.c defs.h
9 cc -c main.c
10
11 kbd.o: kbd.c defs.h command.h
12 cc -c kbd.c
13
14 command.o: command.c defs.h command.h
15 cc -c command.c
16
17 clean:
18 rm -f *.o editor
```

- Ignoring errors:

```
1 clean:
2     -rm -rf *.o editor
```

The leading `-` before `rm` is Makefile syntax, not part of the shell command. It tells make: Run this command, but ignore errors if it fails.

- **What makefile contains:** Makefiles contain five kinds of things: explicit rules, implicit rules, variable definitions, directives, and comments. Rules, variables, and directives are described at length in later chapters
 1. **An explicit rule** says when and how to remake one or more files, called the rule's targets. It lists the other files that the targets depend on, called the prerequisites of the target, and may also give a recipe to use to create or update the targets.
 2. **An implicit rule** says when and how to remake a class of files based on their names. It describes how a target may depend on a file with a name similar to the target and gives a recipe to create or update such a target
 3. **A variable definition** is a line that specifies a text string value for a variable that can be substituted into the text later. T
 4. **A directive** is an instruction for make to do something special while reading the makefile. These include:
 - reading another makefile
 - Deciding (based on the values of variables) whether to use or ignore a part of the makefile
 - Defining a variable from a verbatim string containing multiple lines (
- **Comments:** `#` in a line of a makefile starts a comment. It and the rest of the line are ignored, except that a trailing backslash not escaped by another backslash will continue the comment across multiple lines. A line containing just a comment (with perhaps spaces before it) is effectively blank, and is ignored. If you want a literal `#`, escape it with a backslash (e.g., `\#`). Comments may appear on any line in the makefile, although they are treated specially in certain situations.

You cannot use comments within variable references or function calls: any instance of `#` will be treated literally (rather than as the start of a comment) inside a variable reference or function call.

Within a define directive, comments are not ignored during the definition of the variable, but rather kept intact in the value of the variable. When the variable is expanded they will either be treated as make comments or as recipe text, depending on the context in which the variable is evaluated.

- **Splitting without adding whitespace:** If you need to split a line but do not want any whitespace added, you can utilize a subtle trick: replace your backslash/newline pairs with the three characters dollar sign, backslash, and newline:

```
1 var := one$\
2 word
```

After make removes the backslash/newline and condenses the following line into a single space, this is equivalent to:

```
1 var := one$ word
```

Then make will perform variable expansion. The variable reference ‘\$ ’ refers to a variable with the one-character name “ ” (space) which does not exist, and so expands to the empty string, giving a final assignment which is the equivalent of

- **Names make looks for:** When make looks for a makefile, it tries the following names, in order
 - GNUmakefile
 - makefile
 - Makefile

GNUmakefile will not be understood by other versions of make. You can specify another name with `-f (--file)`

- **Including makefiles:** The include directive tells make to suspend reading the current makefile and read one or more other makefiles before continuing. The directive is a line in the makefile that looks like this:

```
1 include filenames...
```

Which can contain shell file name patterns. If no files are supplied, nothing is included and no error is printed.

- **The MAKEFILES env variable:** If the environment variable MAKEFILES is defined, make considers its value as a list of names (separated by whitespace) of additional makefiles to be read before the others. This works much like the include directive:
- **How make reads a makefile:** GNU make does its work in two distinct phases. During the first phase it reads all the makefiles, included makefiles, etc. and internalizes all the variables and their values and implicit and explicit rules, and builds a dependency graph of all the targets and their prerequisites. During the second phase, make uses this internalized data to determine which targets need to be updated and run the recipes necessary to update them.

It’s important to understand this two-phase approach because it has a direct impact on how variable and function expansion happens; this is often a source of some confusion when writing makefiles. Below is a summary of the different constructs that can be found in a makefile, and the phase in which expansion happens for each part of the construct.

We say that expansion is immediate if it happens during the first phase: make will expand that part of the construct as the makefile is parsed. We say that expansion is deferred if it is not immediate. Expansion of a deferred construct part is delayed until the expansion is used: either when it is referenced in an immediate context, or when it is needed during the second phase.

A rule is always expanded the same way, regardless of the form:

```
1 immediate : immediate ; deferred
2     deferred
```

That is, the target and prerequisite sections are expanded immediately, and the recipe used to build the target is always deferred. This is true for explicit rules, pattern rules, suffix rules, static pattern rules, and simple prerequisite definitions.

- **One liners:** In a Makefile, a semicolon after the prerequisites lets you put the first recipe command on the same line as the target rule.

```
1 target: prerequisites ; recipe-command
```

For example,

```
1 clean: ; rm -f *.o editor
```

you can still add more recipe lines afterward:

```
1 clean: ; rm -f *.o editor
2     rm -rf build
```

- **Forcing:** FORCE: is a common Makefile trick used to make another target run unconditionally.

```
1 FORCE:
2
3 clean: FORCE
4     rm -f *.o editor
```

FORCE has no prerequisites, and no recipe. Usually no file named FORCE EXISTS. So, when another target depends on it:

```
1 clean: FORCE
```

make tries to build FORCE. Since there is normally no real file named FORCE, make treats FORCE as something that is always out of date. That makes clean out of date too, so clean runs.

- **Special built in target names:**
 - **.PHONY:** The prerequisites of the special target .PHONY are considered to be phony targets. When it is time to consider such a target, make will run its recipe unconditionally, regardless of whether a file with that name exists or what its last-modification time is.

- **.SUFFIXES**: The prerequisites of the special target `.SUFFIXES` are the list of suffixes to be used in checking for suffix rules.
- **.DEFAULT**: The recipe specified for `.DEFAULT` is used for any target for which no rules are found (either explicit rules or implicit rules).

If a `.DEFAULT` recipe is specified, every file mentioned as a prerequisite, but not as a target in a rule, will have that recipe executed on its behalf.

- **.PRECIOUS**: The targets which `.PRECIOUS` depends on are given the following special treatment: if `make` is killed or interrupted during the execution of their recipes, the target is not deleted.
- **.INTERMEDIATE**: The targets which `.INTERMEDIATE` depends on are treated as intermediate files.
- **.NONINTERMEDIATE**: Prerequisites of the special target `.NOTINTERMEDIATE` are never considered intermediate files.

`.NOTINTERMEDIATE` with no prerequisites causes all targets to be treated as not intermediate.

If the prerequisite is a target pattern then targets that are built using that pattern rule are not considered intermediate.

- **.SILENT**: If you specify prerequisites for `.SILENT`, then `make` will not print the recipe used to remake those particular files before executing them. The recipe for `.SILENT` is ignored.

If mentioned as a target with no prerequisites, `.SILENT` says not to print any recipes before executing them.

you want to silence all recipes for a particular run of `make`, use the `-s` or `--silent` option (

- **.IGNORE**: If you specify prerequisites for `.IGNORE`, then `make` will ignore errors in execution of the recipe for those particular files. The recipe for `.IGNORE` (if any) is ignored.

If mentioned as a target with no prerequisites, `.IGNORE` says to ignore errors in execution of recipes for all files. This usage of `.IGNORE` is supported only for historical compatibility. Since this affects every recipe in the makefile, it is not very useful; we recommend you use the more selective ways to ignore errors in specific recipes.

- **.ONESHELL**: If `.ONESHELL` is mentioned as a target, then when a target is built all lines of the recipe will be given to a single invocation of the shell rather than each line being invoked separately.
- **.NOTPARALLEL**: If `.NOTPARALLEL` is mentioned as a target with no prerequisites, all targets in this invocation of `make` will be run serially, even if the `-j` option is given. Any recursively invoked `make` command will still run recipes in parallel

- **Independent and grouped targets**: A normal rule uses a single colon

```
1 a b: input.txt
2    ./build-one-output $@
```

This means

```
1  a: input.txt
2     ./build-one-output $@
3
4  b: input.txt
5     ./build-one-output $@
```

So *a* and *b* are treated as separate targets that merely happen to share the same prerequisites and recipe. These are **independent targets**. The automatic variable `$@` is different for each target:

Grouped targets use `&`:

```
1  a b &: input.txt
2     ./generate-both input.txt
```

This means *a* and *b* are produced by one single execution of the recipe.

Make understands that the recipe creates both files together.

So if either *a* or *b* is missing or out of date, Make runs the recipe once, and expects both outputs to be updated.

Variables

- **Variables:** In a Makefile, variables are names that expand to text. They are mostly used to avoid repeating compiler names, flags, filenames, directories, and commands.

```
0  CC = cc
1  CFLAGS = -Wall -Wextra -g
2  OBJS = main.o kbd.o command.o display.o insert.o search.o
   ↪    files.o utils.o
3
4  editor: $(OBJS)
5          $(CC) $(CFLAGS) -o editor $(OBJS)
```

- **Recursive assignment (=):**

```
1  A = $(B)
2  B = hello
3
4  all:
5      echo $(A)
```

With `=`, the variable is expanded later, when it is used.

- **Intermediate assignment (:=):** This prints nothing for A, because B had no value when `A := $(B)` was evaluated.

```
1  A := $(B)
2  B = hello
3
4  all:
5      echo $(A)
```

```
1  A = $(B)
```

Means expand `$(B)` later.

```
1  A := $(B)
```

Means expand `$(B)` now. In ordinary Makefiles, `:=` is often safer and easier to reason about.

- **Appending (+=):** You can add more text to a variable.

```

0  CFLAGS = -Wall
1  CFLAGS += -Wextra
2  CFLAGS += -g

```

- **Conditional assignment (?:=):**

```

1  CC ?= cc

```

Sets CC to cc only if CC does not already have a value. This lets users override it from the command line

```

1  make CC=clang

```

- **Automatic variables:** Make also has special variables that are only meaningful inside recipes.

Variable	Meaning
\$@	The target name.
\$<	The first prerequisite.
\$^	All prerequisites, with duplicates removed.
\$+	All prerequisites, preserving duplicates and order.
\$?	Prerequisites newer than the target.
\$*	The stem matched by a pattern rule.
%	The member name, only for archive targets like lib.a(file.o)

```

1  editor: main.o kbd.o command.o
2      cc -o $@ $^

```

Expands roughly to

```

1  cc -o editor main.o kbd.o command.o

```

- **Directory file variants:**

Variable	Meaning
$\$(@D)$	Directory part of $\$@$.
$\$(@F)$	File part of $\$@$.
$\$(<D)$	Directory part of $\$(<$.
$\$(<F)$	File part of $\$(<$.
$\$(^D)$	Directory parts of $\$(^$.
$\$(^F)$	File parts of $\$(^$.

Patterns and substitutions

- **Patterns:** A pattern is a way to describe a family of filenames using %

The % character means: Match some nonempty or empty part of the filename, then reuse that matched part elsewhere.

That matched part is called the stem.

For example,

```
1  %.o: %.c
2      gcc -c $< -o $@
```

So if Make needs main.o, it matches:

```
1  %.o
```

against

```
1  main.o
```

The stem is therefore **main**. Make then substitutes that stem in the prerequisite pattern %.c

- **Pattern rules:** A pattern rule uses % in the target. This is an implicit rule.
- **Static pattern rules:** A static pattern rule also uses %, but only for a fixed list of targets:

```
1  targets ...: target-pattern: prereq-patterns ...
2      recipe
3      ...
```

For example,

```
1  objects = main.o lexer.o parser.o
2      $(objects): %.o: %.c
3      gcc -c $< -o $@
```

This applies only to the object files listed in the variable **objects**.

- **Substitutions:** In Make, substitution usually means transforming one list of words into another list of words.

The most common case is:

```
1 sources = main.c lexer.c parser.c
2 objects = $(sources:.c=.o)
```

This changes

```
1 main.c lexer.c parser.c
```

Into

```
1 main.o lexer.o parser.o
```

So `$(sources:.c=.o)` means: In each word of `$(sources)`, replace the suffix `.c` with `.o`.

- **Substitution references:** The basic form is

```
1 $(variable:from=to)
```

It only replaces **from** when it appears at the **end** of a word.

- **Pattern substitution references:** There is also a pattern form using `%`

```
1 $(variable:pattern=replacement)
```

```
1 sources = src/main.c src/lexer.c src/parser.c
2 objects = $(sources:src/%.c=build/%.o)
```

Produces

```
1 build/main.o build/lexer.o build/parser.o
```

- **patsubst:** The full function form is

```
1 $(patsubst pattern,replacement,text)
```

```
1 sources = src/main.c src/lexer.c src/parser.c
2 objects = $(patsubst src/%.c,build/%.o,$(sources))
```

Also produces

```
1 build/main.o build/lexer.o build/parser.o
```

- **subst:** There is also a different function called subst:

```
1 $(subst from,to,text)
```

Unlike patsubst, this performs a plain textual replacement

```
1 text = hello.c old.c.backup
2 result = $(subst .c,.o,$(text))
```

Result:

```
1 hello.o old.o.backup
```

This replaces .c wherever it appears, not just at the end of a word.

Make functions

- **filename functions:**

```
1 $(dir names...)
```

Extracts the directory part.

```
1 $(dir src/main.c) # src/
```

```
1 $(notdir names...)
```

Extracts the file name without the directory.

```
1 $(suffix names...)
```

Extracts the file suffix

```
1 basename names...
```

Removes the suffix.

```
1 $(addsuffix suffix, names...)
```

Adds a suffix to each word.

```
1 $(addprefix prefix,names...)
```

Adds a prefix to each word.

```
1 $(wildcard pattern...)
```

Finds existing files matching a shell-style pattern.

- **List functions:** Make treats many variables as whitespace-separated word lists.

```
1 $(word n,text)
```

gets the *n*th word.

```
1 $(words text)
```

counts words.

```
1 $(firstword text)
2 $(lastword text)
```

gets the first or last word

```
1 wordlist start,end,text
```

gets a range of words

```
1 $(filter pattern...,text)
```

keeps only words matching a pattern

```
1 $(filter-out pattern..., text)
```

removes words matching a pattern.

```
1 $(sort list)
```

sorts words and removes duplicates

- **Conditional functions:** Useful for configurable builds

```
1 $(if condition, then-part, else-part)
```

```
1 $(or a,b,c)
```

returns the first nonempty argument

```
1 $(and a,b,c)
```

returns the last argument if all are nonempty; otherwise empty

- **Shell function:**

```
1 $(shell command)
```

runs a shell command and captures its output. Be careful with `$(shell ...)`: it runs during Makefile expansion, not as a normal recipe command.

- **Strip**

```
1 $(strip text)
```

removes leading / trailing whitespace and collapses internal whitespace.

- **eval:**

```
1 $(eval text)
```

takes expanded text and reads it as makefile code.

Conditionals

- **Conditionals:** Make conditionals let the makefile itself choose which text to include before make runs recipes.

They are evaluated by make, not by the shell. That means they affect which variables, rules, prerequisites, and recipe lines exist in the parsed makefile.

```
1 ifeq ($(VAR),value)
2     # used if VAR expands to value
3 else
4     # used otherwise
5 endif
```

```
1 ifneq ($(VAR),value)
2     # used if VAR does not expand to value
3 endif
```

```
1 ifdef VAR
2     # used if VAR has a non-empty value
3 endif
```

```
1 ifndef VAR
2     # used if VAR is empty or undefined
3 endif
```

User-defined functions

- functions:

```
1  define function_name
2      ...
3  endif
```

Call with

```
1  $(call function_name,arg1,arg2,...)
```

```
1  $(i)
```

References the *i*th argument given to the function.

More on make

- **Silencing commands:** In a make recipe, putting @ before a command tells make: Run this command, but do not print the command itself before running it.

```
1 hello:
2     @echo "hello"
```

- **More prefixes:**

```
1 @command    # do not print the command
2 -command    # ignore errors from the command
3 +command    # run even under some recursive/dry-run conditions
```

They can be combined

```
1 clean:
2     @-rm missing-file
```

Header dependencies

- **gcc -MMD -MP**: The GCC flag `-MMD` tells the preprocessor to generate Makefile dependency rules automatically while compiling, but it excludes system header files. This helps automated build systems track exactly when a `.c` file needs to be recompiled due to a change in a local `.h` file.

When you append `-MMD` to your compilation command:

- **Normal Compilation Happens**: It generates the intended object file (e.g., `main.o`).
- **Dependency File Created**: It creates a parallel sidecar file with a `.d` extension (e.g., `main.d`).
- **System Headers Ignored**: It skips system-wide headers (like `<stdio.h>`), mapping only your custom project headers (like `"config.h"`).

For example,

```
1 gcc -MMD -c main.c -o main.o
```

GCC creates `main.o` alongside a new plain text file called `main.d`. If `main.c` relies on a custom header, the generated `main.d` file will contain:

```
1 main.o: main.c config.h
```

Flag	System Headers Included?	Output Behavior
-M	Yes	Prints Makefile rules to standard output; stops compilation.
-MM	No	Prints Makefile rules to standard output; stops compilation.
-MD	Yes	Compiles normally and outputs a <code>.d</code> dependency file.
-MMD	No	Compiles normally and outputs a <code>.d</code> dependency file.

The GCC flag `-MP` tells the preprocessor to add a phony, empty target for each header file dependency in the generated `.d` file

This prevents your build from crashing with a fatal "No such file or directory" error if you delete or rename a header file.

When you combine `-MP` with your dependency generation (usually written as `-MMD -MP`), GCC adds empty rules at the bottom of the `.d` file:

```
1  main.o: main.c config.h
2
3  config.h:
```

Because config.h: has an empty body, if the file is deleted, make simply treats it as an empty target that has already been successfully "built." It will then correctly proceed to recompile main.c, which will now look for the new header or throw a normal compiler error instead of a fatal make error.

```
1  CFLAGS += -MMD -MP
2
3  $(OBJDIR)/%.o: %.c
4      @mkdir -p $(dir $@)
5      $(CC) $(CFLAGS) -c $< -o $@
6
7  -include $(OBJS:.o=.d)
```


Handling dependencies in a build

- **Intro:** In general, handling dependencies in a build means solving four separate problems:
 1. Declare what your project needs.
 2. Locate the dependency on the user's system.
 3. Compile and link against it correctly.
 4. Fail clearly when it is unavailable.
- **Distinguish dependency types:** A dependency can be needed at different stages.
 - **Build dependency:** required to compile the project, such as a compiler, pkg-config, or header files.
 - **Link dependency:** provides symbols your program links against, such as libX11.so.
 - **Runtime dependency:** must exist when the program runs.
 - **Optional dependency:** enables a feature but is not required for the core program.
 - **Development-only dependency:** tests, documentation tools, formatters, or static analyzers.
- **Do not hard-code system paths:** Avoid something like

```
0 CFLAGS += -I/usr/include/imlib2
1 LDFLAGS += -L/usr/lib
```

These paths differ across systems. Instead, use a dependency discovery mechanism. For many C libraries, that is pkg-config

- **pkg-config:** pkg-config is a helper tool used in software development to retrieve the correct compiler and linker flags needed to build applications that depend on external libraries. It saves developers from manually guessing or hard-coding paths to include files (-I) and libraries (-L).

When a library (like glib-2.0 or sqlite3) is installed on a system, it usually provides a metadata file ending in .pc. pkg-config reads this file to find out exactly where the library's header files and binaries are located.

Instead of typing out long build commands, developers embed pkg-config in their build scripts (such as Makefiles) or call it directly using backticks.

```
0 gcc -o my_program my_program.c $(pkg-config --cflags --libs
  ↪ glib-2.0)
```

- **--cflags:** Returns the include paths (where the header files are).
- **--libs:** Returns the library paths and the names of the libraries to link against
- **--exists:** Checks if a specific library or package is available on your system without printing any output to the terminal. It communicates the result entirely through a standard UNIX exit status code.

```
0 pkg-config --cflags imlib2
1 pkg-config --libs imlib2
```

Typical output might be:

```
0 -I/usr/include/imlib2
1 -lImlib2
```

A simple makefile can use something like

```
0 CPPFLAGS += $(shell pkg-config --cflags imlib2 x11)
1 LDLIBS    += $(shell pkg-config --libs imlib2 x11)
```

The build should detect missing dependencies early.

```
0 PKG_CONFIG ?= pkg-config
1
2 DEPS := x11 imlib2
3
4 ifeq ($(shell $(PKG_CONFIG) --exists $(DEPS) && echo yes),)
5 $(error Missing required dependencies: $(DEPS))
6 endif
7
8 CPPFLAGS += $(shell $(PKG_CONFIG) --cflags $(DEPS))
9 LDLIBS    += $(shell $(PKG_CONFIG) --libs $(DEPS))
```

- **Feature detection:** Do not assume that because the system is Linux, a given API or header exists.

```
0 #ifdef HAVE_IMLIB2
1 #include <Imlib2.h>
2 #endif
```

The build system determines whether HAVE_IMLIB2 should be defined:

```
0 CPPFLAGS += -DHAVE_IMLIB2=1
```